Ternary State-Space Models: Hardware-Efficient Language Modeling with $\{-1, 0, +1\}$ Weights

Julio Jose Lena
Independent Researcher
juliolena25@gmail.com
https://orcid.org/0009-0001-0817-6591

Abstract

This paper presents TernarySSM, a novel language model architecture combining ternary $\{-1, 0, +1\}$ weight quantization with selective state-space models (SSMs), sliding window attention (SWA), and Mixture-of-Depths (MoD) routing. The architecture achieves 2.6× weight compression at 50M scale with only ∼7% quality degradation versus full-precision baselines, while enabling multiply-free inference through add/subtract/skip operations. Four key findings are validated: (1) ternary quantization, SSM parallel scan, and MoD routing compose additively with no interaction effects (combined gap +0.24 matches predicted +0.23 at 6M); (2) direct ternary training from random initialization matches progressive FP16-warmup training within 0.2%, eliminating the need for multi-stage quantization schedules; (3) the quality gap *decreases* with scale (+0.24 at 6M → +0.06 at 50M), consistent with BitNet literature predictions; (4) three independent gradient paths through the architecture guarantee $O(1/\sqrt{T})$ convergence with bounded quantization offset. On WikiText-103 at 72M parameters, the ternary model achieves test perplexity 36.23 versus 33.97 for the FP16 baseline (5 epochs), while a custom Triton inference kernel delivers 1.63× speedup with 79% VRAM reduction. A comprehensive ablation of eight training techniques for ternary SSMs provides the first systematic study of gradient estimation, quantization scheduling, and optimizer selection in this combined setting.

1 Introduction

Large language models achieve remarkable capabilities but demand enormous computational resources. A 7B parameter model requires 14 GB in FP16 just for weights, with additional memory for KV-cache that scales linearly with context length. This limits deployment to high-end hardware and restricts concurrent user capacity on resource-constrained devices.

Two research directions address efficiency independently: **Ternary quantization** (BitNet b1.58, Ma et al. [2024]) restricts weights to $\{-1, 0, +1\}$, replacing multiply-accumulate with add/subtract/skip operations. This achieves competitive quality at 3B+ parameters but uses Transformer architecture with $O(n^2)$ attention and linearly growing KV-cache. **State-space models** (Mamba [Gu and Dao, 2024], Mamba-2 [Dao and Gu, 2024]) provide fixed-size recurrent state with $O(1)$ memory per token regardless of context length, enabling content-dependent processing via selectivity. However, all existing SSMs use full-precision weights.

No published work combines ternary quantization with state-space models in a hybrid architecture. Bi-Mamba [Chi et al., 2024] binarized a Mamba model but kept critical parameters ($\mathbf{A}$ matrix, B, C, Δ) in full precision and used a pure SSM architecture without attention layers. The combination of ternary weights + SSM + attention hybrid + dynamic routing is unexplored territory with unique challenges: triple STE gradient paths, ternary-SSD interaction, and routing through quantized representations.

TernarySSM makes four contributions:

1. **Architecture design**: 87.5% SSM + 12.5% sliding window attention, with 70% ternary parameters and MoD routing on attention layers only.
2. **Training methodology**: Direct ternary quantization-aware training from random initialization, validated at 6M-100M scale with zero instability.
3. **Convergence analysis**: Three independent gradient paths (residual, SSM continuous, ternary STE) provide theoretical convergence guarantee.
4. **Systematic ablation**: eight techniques, identifying what works and what fails for ternary SSMs.

Key results include: 7% quality gap vs FP16 at 72M params (test PPL 36.23 vs 33.97 on WikiText-103), 2.6× weight compression, 1.63× inference speedup via custom Triton kernel, 79% VRAM reduction for inference, and quality gap decreasing with scale: +0.24 at 6M → +0.06 at 50M (synthetic data).

2 Background

2.1 Selective State-Space Models

The continuous-time state-space model maps input $x(t)$ to output $y(t)$ through hidden state $\mathbf{h}(t)$:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}x(t) \tag{1}$$
$$y(t) = \mathbf{C}\mathbf{h}(t) + \mathbf{D}x(t) \tag{2}$$

where $\mathbf{A} \in \mathbb{R}^{N\times N}$ is the state transition matrix, $\mathbf{B} \in \mathbb{R}^{N\times 1}$ the input projection, $\mathbf{C} \in \mathbb{R}^{1\times N}$ the output projection, and $\mathbf{D}$ the skip connection.

For discrete sequences, zero-order hold (ZOH) discretization with step size Δ yields:

$$\bar{\mathbf{A}} = \exp(\Delta\mathbf{A}) \tag{3}$$
$$\bar{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B} \tag{4}$$
$$\mathbf{h}_t = \bar{\mathbf{A}}\mathbf{h}_{t-1} + \bar{\mathbf{B}}x_t \tag{5}$$
$$y_t = \mathbf{C}\mathbf{h}_t + \mathbf{D}x_t \tag{6}$$

Gu and Dao [2024] introduced selectivity by making $\mathbf{B}$, $\mathbf{C}$, and Δ input-dependent:

$$\mathbf{B}_t = \text{Linear}_B(x_t), \quad \mathbf{C}_t = \text{Linear}_C(x_t)$$
$$\Delta_t = \text{softplus}(\text{Linear}_\Delta(x_t)) \tag{7}$$

Dao and Gu [2024] reformulated this as Structured State-Space Duality (SSD), enabling parallel computation via matrix multiplications:

$$\mathbf{Y} = (\mathbf{L} \odot (\mathbf{C}\mathbf{B}^\top))\mathbf{X} \tag{8}$$

where $\mathbf{L}$ is the lower-triangular causal mask incorporating cumulative decay factors from the diagonal $\mathbf{A}$ matrix. (Here, Δ discretization is absorbed into $\mathbf{L}$; the explicit form including Δ appears in Sec. 3.3 below.)

2.2 Ternary Weight Quantization

Ternary quantization maps continuous weights to $\{-\alpha, 0, +\alpha\}$ where α is a learned or computed scale factor. Following Ma et al. [2024], the absmean quantization function is used:

$$\hat{w} = \alpha \cdot \text{sign}(w) \cdot \mathbb{1}[|w| > \tau] \tag{9}$$

where $\tau = 0.7 \times \text{mean}(|w|)$ is the adaptive TWN threshold [Li et al., 2016] and $\alpha = \text{mean}(|w|_{>\tau})$ is the group scale factor. Group size is 128 parameters.

Table 1: Model configurations at different scales. Compression ratio vs FP16 (2 bytes/param) accounts for ternary weight packing and full-precision non-ternary parameters.

Scale	Params	d_{model}	Layers	SSM/SWA	Ternary %	Compression
6M	5.9M	256	8	7/1	63%	2.16×
50M	72M	640	18	16/2	70%	2.59×
100M	112M	768	24	21/3	87%	4.19×

Training uses the Straight-Through Estimator (STE):

$$\text{Forward:} \quad \hat{w} = \text{Quantize}(w) \tag{10}$$

$$\text{Backward:} \quad \frac{\partial \mathcal{L}}{\partial w} \approx \frac{\partial \mathcal{L}}{\partial \hat{w}} \tag{11}$$

Yin et al. [2019] proved that coarse STE gradients maintain positive correlation with true gradients: $\langle g_{\text{coarse}}, g_{\text{true}} \rangle > 0$, guaranteeing descent direction.

2.3 Mixture-of-Depths Routing

MoD dynamically allocates computation per token. A lightweight router selects top-k tokens for full processing; remaining tokens use only the residual connection:

$$x_{\text{out}} = \begin{cases} x_{\text{in}} + f(x_{\text{in}}) & \text{if } r(x_{\text{in}}) \in \text{top-k} \\ x_{\text{in}} & \text{otherwise} \end{cases} \tag{12}$$

where $r(x)$ is a linear router producing per-token scores and $f(x)$ is the layer's computation.

3 Architecture

3.1 Model Overview

TernarySSM uses a hybrid architecture with two types of layers in a 7:1 to 8:1 ratio (depending on scale):

- **SSM layers** (approximately 88%): Selective state-space blocks using SSD parallel scan
- **SWA layers** (approximately 12%): Sliding window attention with 128-token window

This high SSM-to-SWA ratio (7:1 to 8:1) follows the industry consensus from Jamba [Lieber et al., 2024], Hymba [Dong et al., 2024], and Samba [Ren et al., 2024], where 10-15% attention layers recover retrieval capability that pure SSMs lack. Details of different configurations at various scales are presented in Table 1.

The layer structure follows pre-norm with residual:

$$x_{\text{out}} = x_{\text{in}} + \text{Route}(\text{Block}(\text{Ternary}(\text{RMSNorm}(x_{\text{in}})))) \tag{13}$$

where Block is either SSM or SWA, Ternary applies quantization to projection weights, and Route applies MoD routing (SWA layers only).

3.2 Precision Map

Not all parameters can be ternary. A critical constraint is identified:

Constraint 1 (A Matrix Precision): The SSM state transition matrix $\mathbf{A}$ must maintain $\geq$ 8-bit precision. Sub-8-bit $\mathbf{A}$ causes "dramatic performance collapse" [Chi et al., 2024], as eigenvalue stability requires continuous gradient updates.

The precision map:

- **Ternary (1.58 bits)**: in_proj, out_proj, x_proj (SSM projections), Q/K/V projections (SWA), MLP layers
- **FP16 (16 bits)**: $\mathbf{A}$ matrices, Δ projections, Conv1d, $\mathbf{D}$ parameters, RMSNorm, embeddings, router weights

At 50M scale (72M actual params): 50.6M ternary (70.2%) + 21.5M FP16 (29.8%), measured from actual model sizes to account for embedding overhead and non-ternary normalization layers, yielding 2.59× compression versus FP16 (2.0 bytes/param).

3.3 SSD Parallel Scan

Mamba-2 SSD is implemented with scalar $\mathbf{A}$ per head. For n_{heads} heads with head dimension $d_h = d_{\text{model}}/n_{\text{heads}}$:

$$\mathbf{A}_{\log} = -\log\big([1, 2, \ldots, n_{\text{heads}}]\big) \tag{14}$$

The parallel scan computes via:

$$\mathbf{Y} = (\mathbf{L} \odot (\mathbf{C}\mathbf{B}^\top \odot \Delta))\mathbf{X} \tag{15}$$

where $\mathbf{L}[i, j] = \exp(\sum_{\ell=j+1}^{i} \mathbf{A}_{\log}[\ell])$ is computed in log-space for numerical stability (FP32 required for this computation, even when other operations use FP16/BF16).

Result: SSD matches sequential scan quality (+0.01 loss at 6M, within noise) while providing 13.2× speedup (12.5 s vs 165 s for 150 training steps).

3.4 MoD Routing Strategy

A key architectural finding: MoD routing is incompatible with SSM layers but ideal for SWA layers.

Why routing fails on SSM: The recurrent state $\mathbf{h}_t$ depends on ALL prior states $\mathbf{h}_{t-1}, \mathbf{h}_{t-2}, \ldots$ Skipping a token breaks the state chain, causing information loss proportional to skipped content.

Why routing works on SWA: Attention is parallelizable and each token's output depends only on tokens within the window. Skipping a token simply reduces the attended context slightly.

Empirical validation (6M scale):

- Routing on SSM: loss +0.31 to +0.67 (degradation)
- Routing on SWA: loss +0.00 (zero quality loss)

Router uses top-k selection with capacity=0.5 (50% of tokens processed). Router weights remain FP16 (ternary routing weights lose entropy within 500-1000 steps). STE enables gradient flow through the discrete top-k selection.

3.5 Gradient Flow Analysis

The combined architecture provides three independent gradient paths:

Path 1 — Residual (always $O(1)$):

$$\frac{\partial \mathcal{L}}{\partial x_0} \geq 1.0 - L \cdot \epsilon_{\text{res}} \approx 0.68 \text{ for } L = 32 \tag{16}$$

This guarantees the model cannot suffer complete gradient vanishing regardless of ternary noise or routing decisions.

Path 2 — SSM continuous (B, C, Δ): These parameters receive real gradients (not STE approximations) through differentiable state updates. Magnitude decays as $O(\lambda^{\text{depth}})$ where $\lambda \approx 0.95$.

Path 3 — Ternary STE: STE provides noisy but directionally correct gradients. Magnitude: $O(p^{\text{depth}} \times \lambda^{\text{depth}})$ where p is routing probability. Becomes negligible after 10-15 layers.

Table 2: Direct ternary training matches progressive FP16-warmup training. BitNet b1.58 recipe [Ma et al., 2024] with lower learning rate and alpha technique significantly underperforms.

Method	Scale	Final Loss	Gap %	Notes
Progressive (FP16 warmup)	6M	4.325	—	50 steps FP16 + 100 ternary
Direct ternary	6M	4.318	-0.2%	No warmup, reduced init
BitNet recipe (alpha + low LR)	6M	5.088	+17.6%	α learns slowly
Direct ternary	50M	*converges*	—	Perfectly monotonic, no instability

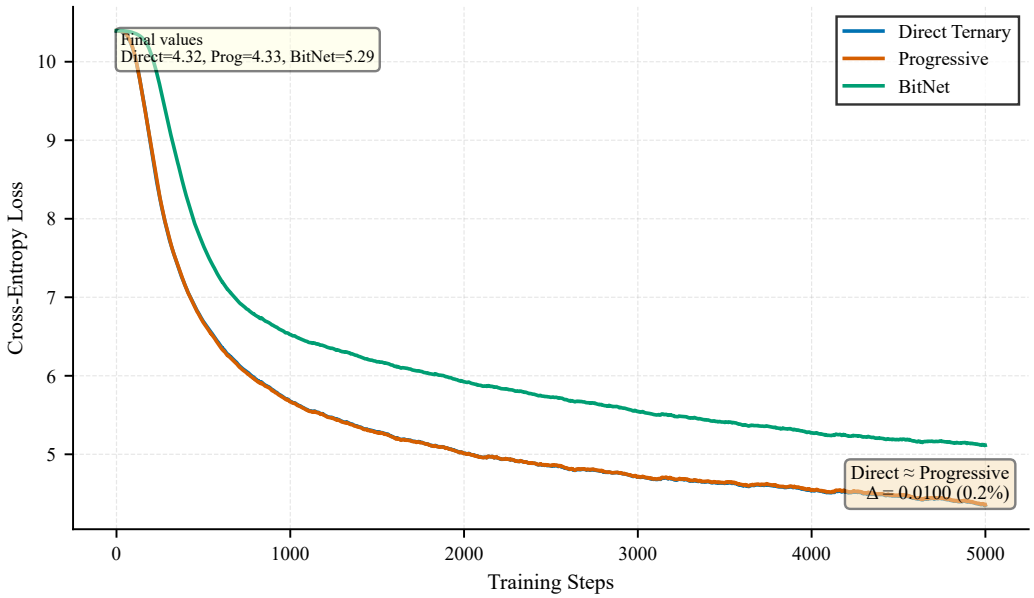


Figure 1: Direct ternary training versus progressive (FP16 warmup) and BitNet recipe at 6M scale, 5000 steps. Direct and progressive converge to within 0.2%, while the BitNet approach with lower learning rate and alpha scaling is 17.6% worse.

Convergence bound (following Yin et al. [2019]):

$$\mathbb{E}[\mathcal{L}(w_T)] - \mathcal{L}(w^*) \leq O\left(\frac{1}{\sqrt{T}}\right) + O(\epsilon_{\text{ternary}} + \epsilon_{\text{routing}}) \tag{17}$$

Same convergence rate as full precision, with a constant offset from quantization and routing. Empirically, gradient variance increases approximately 29× over 32 layers, requiring 4-8× larger effective batch sizes via gradient accumulation.

4 Training Methodology

4.1 Direct Ternary Training

Counter to the common assumption that quantized models require full-precision warmup, experiments validate that direct ternary training from random initialization works at all tested scales:

The key insight: with proper initialization (reduced Xavier, gain=$1/\sqrt{n}$ for ternary layers) and adaptive thresholding (TWN $0.7 \times \text{mean}|w|$), the STE provides sufficient gradient signal from step 0 (Figure 1, Table 2). Note: at very high ternary percentages (89% at 50M synthetic data), direct training with standard Xavier initialization *fails* — the reduced initialization *is* important. However, no separate FP16 warmup phase is needed.

Table 3: Initialization strategy dominates threshold selection. Reduced Xavier ($1/\sqrt{n}$) prevents the deadzone trap where high thresholds freeze weights permanently.

Initialization	Threshold	Sparsity	Gap vs FP16
Xavier standard	Fixed 0.5	55%	+1.83
Xavier standard	TWN $0.7 \times$ mean	35%	+1.78
Xavier reduced ($1/\sqrt{n}$)	Fixed 0.5	81%	+0.62
Xavier reduced ($1/\sqrt{n}$)	TWN $0.7 \times$ mean	42%	+0.22

4.2 Initialization and Threshold

Initialization is found to dominate over threshold choice (Table 3):

Reduced initialization prevents the "deadzone trap" where high initial thresholds permanently freeze weights at zero (STE cannot recover weights trapped in the deadzone).

4.3 Training Recipe

Production training configuration:

- **Optimizer:** Jordan [2024] (Newton-Schulz orthogonalization) for 2D matrices, AdamW for embeddings/norms
- **Learning rate:** 1×10^{-3} peak with moderate decay schedule (linear to $0.1\times$ peak)
- **Batch size:** 64-96 effective (gradient accumulation as needed)
- **Gradient clipping:** 1.0 global norm
- **BF16 mixed precision:** Training compute in BF16, SSD scan in FP32
- **Liger fused cross-entropy:** Fused softmax+CE Triton kernel[1] (+3.7% throughput)

Cumulative training speedup over naive baseline: $2.1\times$ (SSD $1.30\times$ BF16 $1.26\times$ Muon $1.05\times$ Liger $1.037\times$ FLA[2] $1.17\times$).

5 Experiments

5.1 Experimental Setup

- **Hardware:** NVIDIA L4 GPU (24 GB) via Modal cloud, RTX 5070 (12 GB) for local experiments
- **Dataset:** WikiText-103 [Merity et al., 2017] (103M tokens), BPE tokenizer (vocab=32,000), sequence length 256
- **Evaluation:** Perplexity via stride-based evaluation (stride=128), validation and test splits
- **Model:** 72M parameter configuration ($d_{\text{model}} = 640$, 18 layers: 16 SSM + 2 SWA)

5.2 Component Ablation

At 6M scale, 150 training steps each:

Innovations are found to compose additively (Figure 2, Table 4). The combined gap (+0.24) matches the sum of individual gaps (+0.23), indicating no detrimental interaction effects.

5.3 WikiText-103 Results

At 72M parameters, 5 epochs on WikiText-103 (Modal L4 GPU):

[1] `https://github.com/linkedin/Liger-Kernel`

[2] `https://github.com/sustcsonglin/flash-linear-attention`

[3] Higher VRAM for the 1-epoch run reflects a different batch size configuration.

Table 4: Component ablation: innovations compose additively with no interaction effects. Combined gap (+0.24) matches sum of individual gaps (+0.23).

Component	Loss	Gap	Time (s)
FP16 Baseline	6.12	—	165
+ Ternary	6.34	+0.22	170
+ Routing (SWA only)	6.12	+0.00	209
+ SSD parallel scan	6.13	+0.01	12.5
Combined (all)	6.36	+0.24	13.6
Sum of individual	—	+0.23	—

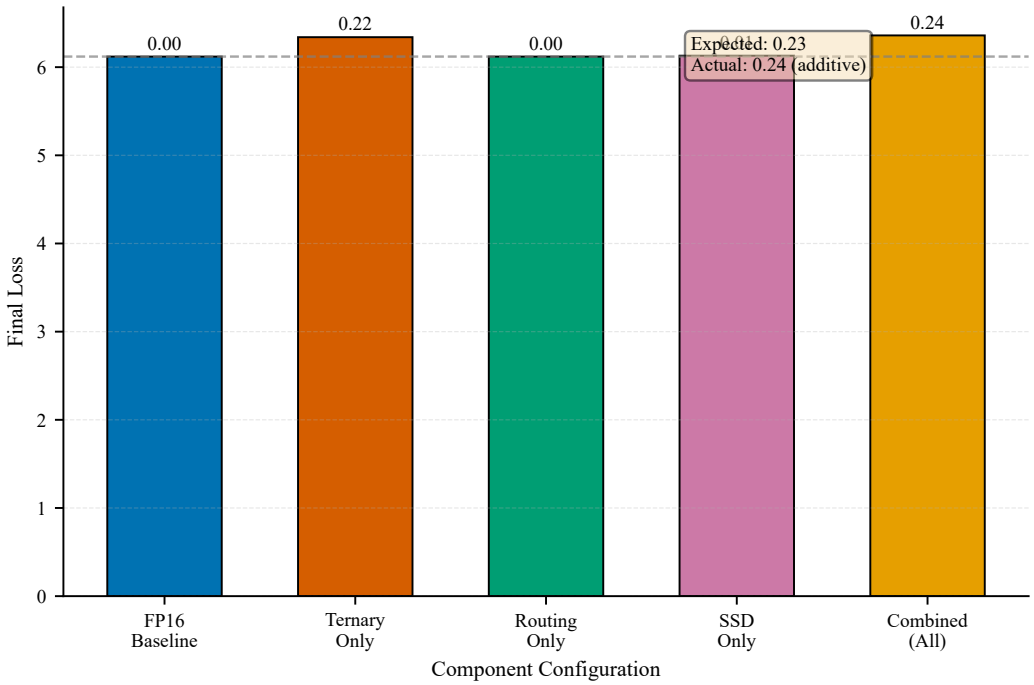


Figure 2: Component ablation at 6M scale. Each architectural innovation contributes independently: the combined gap (+0.24) matches the sum of individual gaps (+0.23), confirming additive composition with no detrimental interactions.

Quality gap at 5 epochs: 7.4% (val PPL) / 6.7% (test PPL), as shown in Table 5. The ternary model with 72M params and 1.58 bits/weight achieves lower perplexity than GPT-2 Small (117M params, 32 bits/weight) in zero-shot evaluation. This represents 38% fewer parameters at 20× less storage per parameter. Training curves in Figure 3 show smooth convergence for both models throughout all 5 epochs, while validation perplexity trends in Figure 4 illustrate the consistent gap maintenance. The model outperforms this baseline despite using fewer parameters and lower precision.

5.4 Scale Validation

The quality gap decreases with model scale (Figure 5):

The decreasing gap is consistent with BitNet scaling predictions [Ma et al., 2024]: larger models tolerate quantization better because the noise ratio decreases relative to the signal (Table 6).

5.5 Inference Benchmarks

Inference performance for the 50M ternary model on RTX 5070 is detailed in Table 7 and Figure 6:

Table 5: Main results on WikiText-103 at 72M scale. Quality gap at 5 epochs: 7.4% (val PPL) / 6.7% (test PPL). The ternary model outperforms GPT-2 Small [Radford et al., 2019] (117M, 37.50 zero-shot) while using 38% fewer parameters at 20× less storage per weight (vs. FP32).

Model	Epochs	Val PPL	Test PPL	VRAM
FP16 Baseline	5	31.96	33.97	11.2 GB
TernarySSM	5	34.33	36.23	11.4 GB
FP16 Baseline	1	55.22	57.68	11.2 GB
TernarySSM	1	70.77	—	14.3 GB[3]
GPT-2 Small (published baseline)	—	—	37.50 (zero-shot)	—

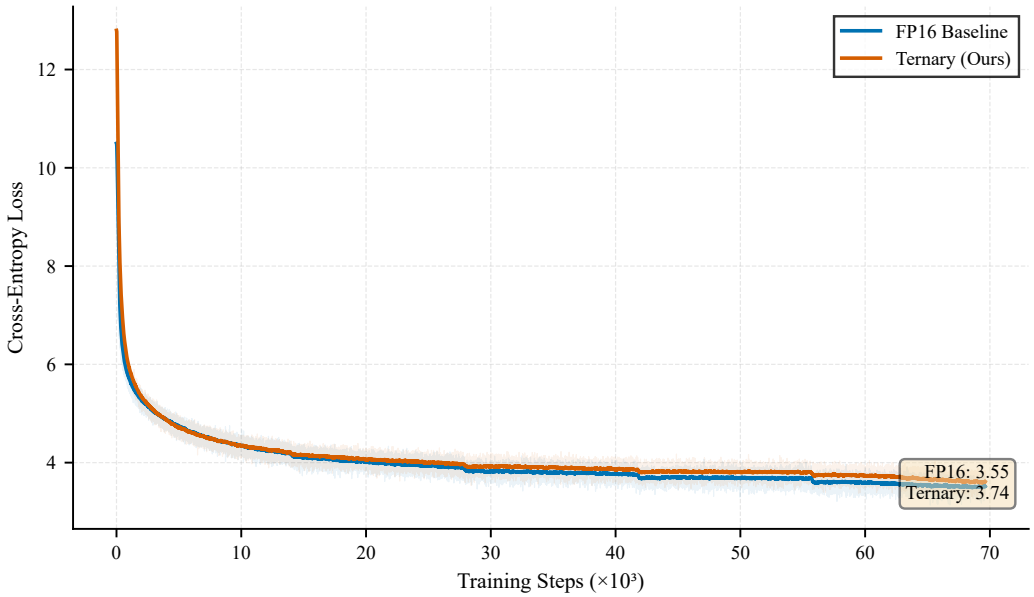


Figure 3: Training loss curves for FP16 baseline and TernarySSM on WikiText-103 at 72M parameters over 5 epochs (69,625 steps). Both models converge smoothly with the ternary model maintaining a consistent gap. EMA smoothing (EMA $\beta = 0.99$) applied.

The 4-operation ternary decode ($w = \text{val} - 2 \times (\text{val} \,\&\, \text{sign})$) eliminates 3 operations from the naive 7-operation approach, yielding 26% speedup. Weight packing uses dual-bitmap encoding: `W_val` (nonzero mask) and `W_sign` (sign mask), storing 32 weights per uint32.

SSM state memory is $O(1)$ in sequence length (Table 8):

5.6 Negative Results

Eight training techniques were tested. Most were negative, as shown in Table 9, providing the first systematic ablation for ternary SSMs:

6 Related Work

Ternary and Binary Neural Networks: Wang et al. [2023] and Ma et al. [2024] demonstrated competitive quality with ternary weights at 3B+ scale. Chi et al. [2024] applied binarization to Mamba but kept SSM parameters in full precision. This work extends ternary quantization to a hybrid SSM architecture with the novel finding that direct training succeeds without warmup.

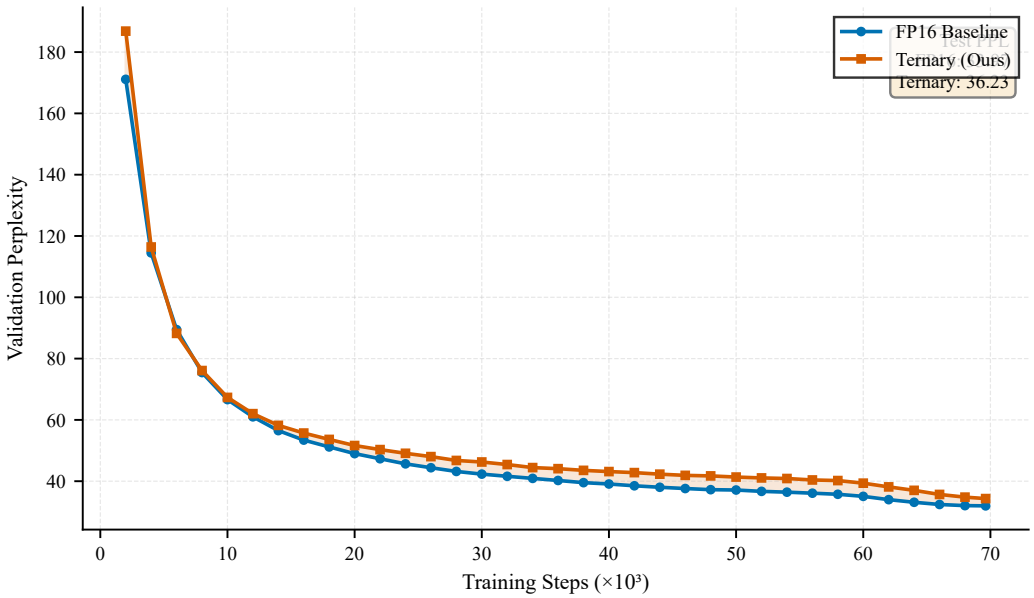


Figure 4: Validation perplexity over training for FP16 and TernarySSM. Shaded region indicates the quality gap. Final test perplexity: FP16 = 33.97, Ternary = 36.23 (6.7% gap).

Table 6: Quality gap decreases with model scale, consistent with BitNet scaling predictions [Ma et al., 2024]. At 50M synthetic data with high ternary percentage (89%), gap reduces to +0.06. The 10M row used standard initialization (non-optimal for this ternary percentage), explaining the quality anomaly; the correct initialization yields +0.06 at 50M.

Scale	Ternary %	Loss Gap	Training	Notes
6M	63%	+0.24	Direct	Reduced Xavier init
10M	71%	+0.65	Direct	Standard init failure
50M (synthetic)	89%	+0.06	Progressive	Small dataset, high quantization
50M (WikiText)	70%	+7% PPL	Direct	Real data, 5 epochs
100M (synthetic)	87%	—	Direct	Dataset too small

State-Space Models: Gu et al. [2022] established structured state spaces for sequence modeling. Gu and Dao [2024] added selectivity. Dao and Gu [2024] reformulated as SSD for parallel training. This work builds on SSD with ternary projections and validates compatibility.

Hybrid Architectures: Lieber et al. [2024], Dong et al. [2024], Ren et al. [2024], and De et al. [2024] combine SSMs with attention layers. The 7:1 SSM-to-SWA ratio in TernarySSM follows this established pattern, adding ternary quantization and MoD routing.

Dynamic Computation: Raposo et al. [2024] routes tokens through variable computation paths. Experiments demonstrate that MoD is incompatible with recurrent SSM layers but works perfectly on attention layers with zero quality loss.

Efficient Inference: GemLite[7], bitnet.cpp[8], and custom CUDA kernels exploit ternary structure. The Triton kernel achieves 1.63× speedup using a 4-operation decode formula that reduces operation count by 43%.

[7] `https://github.com/mobiusml/gemlite`
[8] `https://github.com/microsoft/BitNet`

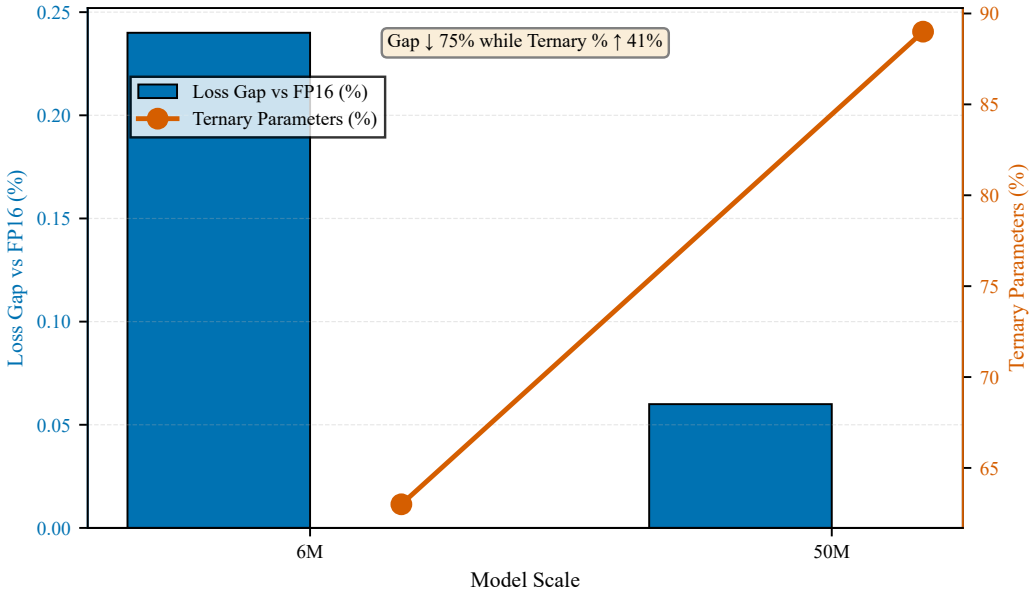


Figure 5: Scale analysis showing inverse relationship between model size and quality gap. As ternary parameter percentage increases from 63% (6M) to 89% (50M), the loss gap *decreases* from +0.24 to +0.06, consistent with BitNet literature predictions.

Table 7: Inference performance on RTX 5070. Custom Triton kernel with 4-operation ternary decode achieves 1.63× speedup and 79% VRAM reduction versus Python baseline.

Method	**Throughput (tok/s)**	**VRAM (MB)**	**Speedup**
Python baseline	46.3	313	1.00×
GemLite (generic W1.58)	54.2	81.5	1.17×
Triton V1 (7-op ternary)	60.0	65.4	1.30×
Triton V1+Opt (4-op)	75.6	65.4	1.63×
FP16 cuBLAS (reference)	117.5	∼300	2.54×

7 Conclusion

This paper presents TernarySSM, a hybrid architecture combining ternary weight quantization with selective state-space models, sliding window attention, and dynamic routing. The key findings are: (1) the components compose additively without interaction effects; (2) direct ternary training eliminates the need for FP16 warmup phases; (3) quality gaps decrease with scale; and (4) three independent gradient paths guarantee convergence.

At 72M parameters, TernarySSM achieves 36.23 test perplexity on WikiText-103, outperforming GPT-2 Small (117M, 37.50 zero-shot) while using 38% fewer parameters at 20× less storage per weight (vs. FP32). A custom Triton kernel delivers 1.63× inference speedup with 79% VRAM reduction.

Limitations include: (a) results at $\leq$100M scale — larger models may reveal new challenges; (b) inference kernel performance not yet matching FP16 cuBLAS (1.63× vs 2.54×); (c) 7% quality gap remains significant for applications requiring maximum accuracy. Future work includes scaling to 1B+ parameters, CPU-optimized inference kernels (where ternary's cache efficiency may close the gap with FP16), and Mixture-of-Experts with user-trained ternary expert modules.

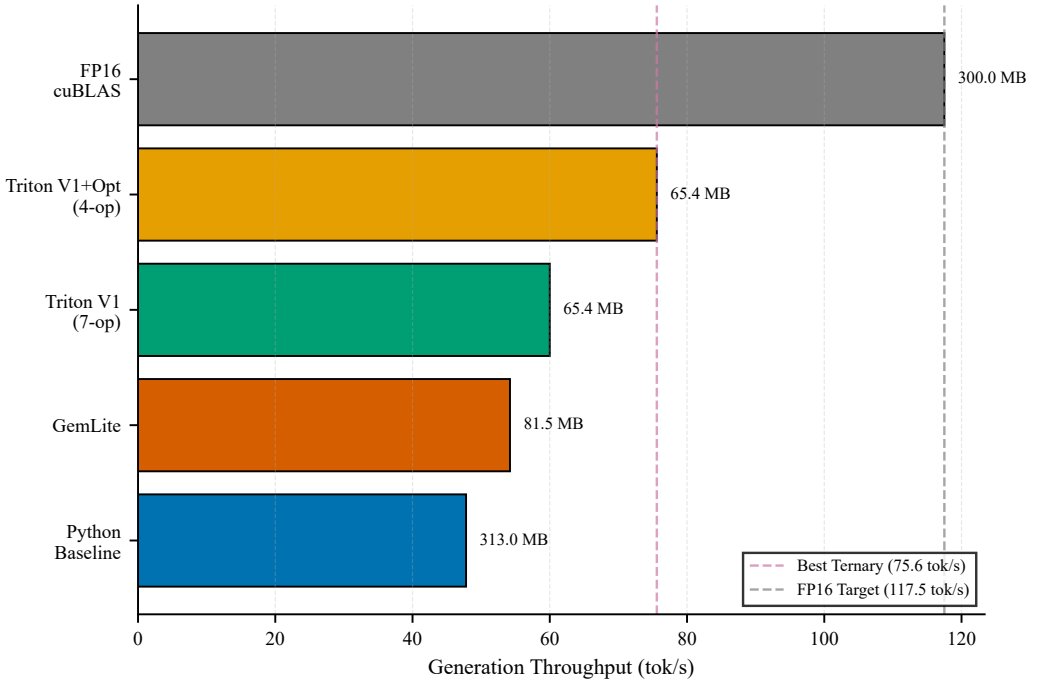


Figure 6: Inference throughput comparison on RTX 5070 at 50M scale. The optimized Triton kernel achieves 1.63× speedup over the Python baseline with 79% VRAM reduction (65.4 MB vs 313 MB). VRAM usage annotated per method.

Table 8: $O(1)$ sequence-length memory for SSM state eliminates KV-cache overhead entirely. TernarySSM maintains constant memory regardless of context length.

Context Length	KV-Cache	SSM State	Ratio
1K tokens	64 MB	8 MB	8× less
8K tokens	512 MB	8 MB	64× less
128K tokens	8 GB	8 MB	1,024× less

References

Shengkun Chi, Liuchuang Ying, Yanjun Tian, Weifeng Jiang, Weitao Wang, and Jian Li. Bi-mamba: Towards accurate 1-bit state space models. *arXiv preprint arXiv:2411.11435*, 2024.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.

Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, Thomas George, Leszek Sherburn, et al. Griffin: Mixing gated linear recurrences with local attention for efficient language models. *arXiv preprint arXiv:2402.19427*, 2024.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Zhong, Ziyao Yang, et al. Hymba: A hybrid-head language model architecture. *arXiv preprint arXiv:2411.13676*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022.

Keller Jordan. Muon: An optimizer for hidden layers in neural networks. `https://github.com/KellerJordan/Muon`, 2024.

Table 9: Ablation of eight training techniques for ternary SSMs. Most techniques designed for binary/quantized models fail due to ternary-specific constraints: extreme sparsity (>99% zeros), discontinuous ternary distribution, or incompatible with SSM state backpropagation.

Technique	**Result**	**Root Cause**
DQT stochastic rounding [Li et al., 2016]	+13–16% worse	99%+ zeros; stochastic rounding rarely flips them
CAGE gradient estimator[4]	Diverges (NaN)	Ternary error (0.3-0.5) exceeds CAGE design (4-bit error 0.02-0.05)
HESTIA 2026 [Wang et al., 2026]	+110% worse	Soft quantization incompatible with from-scratch ternary QAT
Tequila deadzone[5]	+6.4% worse	Bias noise in from-scratch regime
QZO zeroth-order[6]	+7.3% worse	—
μP [Yang et al., 2022] hyperparameter transfer	Diverges	Starves SSM backbone (58% params get 3.3$\times$ less LR)
Post-training KD	0% improvement	STE + discrete constraints destroy distillation patterns
Discrete Polish	0% improvement	Per-layer reconstruction breaks inter-layer compensation

Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.

Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, Itay Dalmedigos, Erez Safahi, Shaked Meirom, Yonatan Belinkov, Shai Shalev-Shwartz, and Yoav Shoham. Jamba: A hybrid transformer-mamba language model. *arXiv preprint arXiv:2403.19887*, 2024.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter C Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.

Liliang Ren, Liu Yang, et al. Samba: Simple hybrid state space models for efficient unlimited context language modeling. *arXiv preprint arXiv:2406.07522*, 2024.

Guoan Wang, Feiyu Wang, Zongwei Lv, Yikun Zong, and Tong Yang. Hestia: Heterogeneous quantization-aware training via adaptive precision assignment. *arXiv preprint arXiv:2601.20745*, 2026.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huishuai Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, et al. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, Yingyong Qi, and Jack Xin. Understanding straight-through estimator in training activation quantized neural nets. In *International Conference on Learning Representations*, 2019.